

Итоговый проект по дисциплине «Разработка прототипов программных решений»

В этом курсе вам предстоит работать над групповыми проектами.

Работа делится на 2 этапа:

- бэкенд — создаёте серверную часть приложения;
- фронтенд — разрабатываете пользовательский интерфейс.

Преподаватели проверят вашу работу после каждого этапа.

Как выбрать проект

Внимательно изучите описание проектов и выберите тот, который подойдёт по уровню сложности и сфере интересов:

- Новичкам подойдут проекты базовой сложности, например интернет-магазин или To-Do приложение.
- Опытным студентам лучше выбрать проект сложнее, например социальная сеть, систему бронирования или учебную платформу.

Все предложенные проекты позволяют использовать React Router, Context API, реализовать взаимодействие с API, и навыки оптимизации приложения.

Успеха в работе над проектом!

Описание проектов

1. Интернет-магазин

Описание: приложение, где пользователи могут просматривать товары, добавлять их в корзину и оформлять заказ.

Основные функции:

- Главная страница с рекламными акциями.
- Каталог товаров с фильтрами по категории, цене и рейтингу.
- Страница товара с детальным описанием и отзывами пользователей.
- Корзина с расчётом общей стоимости и функцией изменения количества товаров.
- Оформление заказа с вводом данных пользователя.
- Личный кабинет пользователя с историей просмотров товаров

ИИ-функции:

- На основе истории просмотра товаров, сделайте систему рекомендаций товаров для пользователя.

Дополнительные функции:

- Сохранение состояния корзины в localStorage.
- Административная панель для добавления и редактирования товаров.

Требования к бэкенду:

1. Разработать модель данных для реализации основных функций проекта. Минимальный набор таблиц:
 - товар: идентификатор, наименование, категория, цена, рейтинг, отзывы;
 - категория: идентификатор, наименование категории;
 - отзывы: идентификатор, ссылка на пользователя, отзыв.
 - корзина: идентификатор, товар, ссылка на пользователя;
2. Разработать модель данных для работы сервиса регистрации, авторизации и аутентификации пользователей. Минимальный набор таблиц:
 - пользователь: идентификатор, имя, фамилия, логин, пароль и подобные, уровень прав (администратор, пользователь).

3. Разработать API для работы основных функций:
 - эндпоинты для добавления, редактирования, удаления товаров: фильтры по нужным атрибутам, пагинация;
 - эндпоинты для взаимодействия с корзиной: фильтры по нужным атрибутам, пагинация.
4. Разработать API для регистрации, авторизации и аутентификации.
Эндпоинты: регистрация, аутентификация, смена пароля, обновление токена.
5. Разработать API и модель данных для рекламных акций.

Результат реализации бэкенда:

1. Разработана модель данных и настроены миграции.
2. Создан API для работы с основными функциями. Код запускается и выполняет требования задания. Допускается и приветствуется любое расширение, дополнение и обоснованное улучшение функционала.
3. Разработан набор тестов для проверки всех эндпоинтов API.
4. Развёртывание: создан файл конфигурации Docker Compose для сборки и запуска проекта в контейнере.

2. Приложение для управления задачами (To-Do App)

Описание: приложение, где пользователи могут создавать, редактировать и удалять задачи, а также сортировать их по категориям или статусу.

Основные функции:

- Система авторизации пользователей.
- Создание, редактирование и удаление задач.
- Фильтрация задач по статусу (выполненная/невыполненная) и категориям.
- Отметка выполнения задач.
- Сортировка задач по приоритету и сроку выполнения.

ИИ-функции:

- Создайте систему, которая будет прогнозировать время выполнения задач на основе времени выполнения задач пользователем.

Дополнительные функции:

- Подключение к внешнему API для хранения задач.
- Поддержка тёмной и светлой тем.

Требования к бэкенду:

1. Разработать модель данных для реализации основных функций проекта. Минимальный набор таблиц:
 - задача: идентификатор, название, описание, категория, статус;
 - категория: идентификатор, наименование категории.
2. Разработать модель данных для работы сервиса регистрации, авторизации и аутентификации пользователей: идентификатор, имя, фамилия, логин, пароль.
3. Разработать API для регистрации, авторизации и аутентификации. Эндпойнты: регистрация, аутентификация, смена пароля, обновление токена.
4. Разработать API для работы основных функций:
 - создание, удаление, изменение и получение задач;
 - изменение статуса;
 - пагинация;
 - фильтры по статусам и категориям задач.

Результат реализации бэкенда:

1. Разработана модель данных и настроены миграции.
2. Создан API для работы с основными функциями. Код запускается и выполняет требования задания. Допускается и приветствуется любое расширение, дополнение и обоснованное улучшение функционала.
3. Разработан набор тестов для проверки всех эндпойнтов API.
4. Развёртывание: создан файл конфигурации Docker Compose для сборки и запуска проекта в контейнере.

3. Платформа для блогов

Описание: приложение, которое позволяет пользователям читать, писать и комментировать статьи.

Основные функции:

- Просмотр статей по категориям.

- Страница статьи с возможностью оставлять комментарии.
- Система авторизации и регистрации пользователей.
- Личный кабинет пользователя с управлением его статьями.
- Редактор статей с возможностью сохранения в черновик или публикации.

ИИ-функции:

- Разработайте систему, которая на основе содержания статьи будет определять характер содержания (позитивный, негативный, нейтральный), а также возрастные ограничения для каждой статьи.

Дополнительные функции:

- Система лайков и сбора статистики по статьям.
- Модерация комментариев.

Требования к бэкенду:

1. Разработать модель данных для реализации основных функций проекта. Минимальный набор таблиц:
 - статья: идентификатор, название, контент, категория, привязка к комментариям;
 - категория: идентификатор, наименование категории;
 - комментарии: автор, комментарий, статья.
2. Разработать модель данных для работы сервиса регистрации, авторизации и аутентификации пользователей: идентификатор, имя, фамилия, логин, пароль.
3. Разработать API для регистрации, авторизации и аутентификации. Эндпойнты: регистрация, аутентификация, смена пароля, обновление токена.
4. Разработать API для работы основных функций, отдельно для статей и комментариев: создание, удаление, редактирование, чтение (пагинация, фильтры).

Результат реализации бэкенда:

1. Разработана модель данных и настроены миграции.
2. Создан API для работы с основными функциями. Код запускается и выполняет требования задания. Допускается и приветствуется любое расширение, дополнение и обоснованное улучшение функционала.

3. Разработан набор тестов для проверки всех эндпойнтов API.
4. Развёртывание: создан файл конфигурации Docker Compose для сборки и запуска проекта в контейнере.

4. Система управления бронированием

Описание: приложение для бронирования номеров в отелях.

Основные функции:

- Поиск доступных вариантов бронирования с возможностью фильтрации по дате, локации и стоимости.
- Страница с детальной информацией о бронировании.
- Система авторизации и сохранение списка бронирований пользователя.
- Оформление бронирования с функцией подтверждения.

ИИ-функции:

- Создать рекомендательную систему с альтернативными вариантами на основе характера просмотров объектов пользователей.

Дополнительные функции:

- Реализовать также личный кабинет управляющего.

Требования к бэкенду:

1. Разработать модель данных для реализации основных функций проекта. Минимальный набор таблиц:
 - бронирование: идентификатор, название, дата, локация, цена, описание, статус подтверждения.
2. Разработать модель данных для работы сервиса регистрации, авторизации и аутентификации пользователей: идентификатор, имя, фамилия, логин, пароль.
3. Разработать API для регистрации, авторизации и аутентификации. Эндпойнты: регистрация, аутентификация, смена пароля, обновление токена.
4. Разработать API для работы основных функций бронирования: создание, удаление, редактирование, чтение (пагинация и фильтры).

Результат реализации бэкенда:

1. Разработана модель данных и настроены миграции.
2. Создан API для работы с основными функциями. Код запускается и выполняет требования задания. Допускается и приветствуется любое расширение, дополнение и обоснованное улучшение функционала.
3. Разработан набор тестов для проверки всех эндпойнтов API.
4. Развёртывание: создан файл конфигурации Docker Compose для сборки и запуска проекта в контейнере.

5. Учебная платформа

Описание: приложение для обучения с возможностью просмотра курсов, прохождения уроков и тестирований.

Основные функции:

- Главная страница с каталогом доступных курсов.
- Просмотр уроков в текстовом формате.
- Тесты с автоматической проверкой.
- Личный кабинет с отображением прогресса по курсам.

ИИ-функции:

- Система генерации персональных тестов на основе изученного пользователем материала.

Дополнительные функции:

- Просмотр уроков в видео формате.
- Раздел для преподавателей с возможностью добавления курсов.
- Отзывы о курсах.

Требования к бэкенду:

1. Разработать модель данных для реализации основных функций проекта. Минимальный набор таблиц:
 - курс: идентификатор, название, описание;
 - урок: идентификатор, принадлежность к курсу, название, описание, контент, ссылка на тест;
 - тест: идентификатор, вопрос, варианты ответов, правильный ответ.

2. Разработать модель данных для работы личного кабинета: список курсов пользователя (пользователь, курс, прогресс).
3. Разработать модель данных для работы сервиса регистрации, авторизации и аутентификации пользователей: идентификатор, имя, фамилия, логин, пароль.
4. Разработать API для регистрации, авторизации и аутентификации. Эндпойнты: регистрация, аутентификация, смена пароля, обновление токена.
5. Разработать API, обеспечивающий работу основных функций для администратора, отдельно по курсам, урокам и тестам: создание, удаление, редактирование, чтение.
6. Разработать API, обеспечивающий работу основных функций для студентов:
 - прохождение курса — последовательное освоение контента;
 - навигация по курсу — получение уже освоенного контента.
7. Разработать API, обеспечивающий работу личного кабинета: навигация по курсам пользователя, просмотр оценок за тесты.

Результат реализации бэкенда:

1. Разработана модель данных и настроены миграции.
2. Создан API для работы с основными функциями. Код запускается и выполняет требования задания. Допускается и приветствуется любое расширение, дополнение и обоснованное улучшение функционала.
3. Разработан набор тестов для проверки всех эндпойнтов API.
4. Развёртывание: создан файл конфигурации Docker Compose для сборки и запуска проекта в контейнере.

6. Приложение для отслеживания расходов

Описание: приложение для управления личными финансами, позволяющее пользователям фиксировать доходы и расходы и получать аналитику по ним.

Основные функции:

- Добавление финансовых транзакций (доходов и расходов) с указанием категории.

- Просмотр аналитики с группировкой по категориям и временным периодам.
- Визуализация данных в виде графиков, например круговая диаграмма расходов.
- Сортировка транзакций по дате и категории.

ИИ-функции:

- Добавить учёт доходов.
- Добавить персональные финансовые рекомендации на основе доходов и расходов пользователя.

Дополнительные функции:

- Экспорт данных в файлы формата .csv или .xlsx.
- Напоминания о регулярных платежах.

Требования к бэкенду:

1. Разработать модель данных для реализации основных функций проекта. Минимальный набор таблиц:
 - транзакция: идентификатор, наименование, тип (доход или расход), категория, сумма, дата, ссылка на пользователя;
 - группа: таблица позволяющая объединить пользователей в группы например для учёта семейных транзакций, идентификатор группы, название группы, ссылка на пользователя.
2. Разработать модель данных для работы сервиса регистрации, авторизации и аутентификации пользователей: идентификатор, имя, фамилия, логин, пароль.
3. Разработать API для регистрации, авторизации и аутентификации. Эндпойнты: регистрация, аутентификация, смена пароля, обновление токена.
4. Разработать API, обеспечивающий работу основных функций для транзакций: создание, удаление, редактирование, чтение (пагинация, фильтры по категории и дате).
5. Разработать API, обеспечивающий работу основных функций для работы с группами:
 - создание, удаление, редактирование, чтение;
 - аналитика по группам.

Результат реализации бэкенда:

1. Разработана модель данных и настроены миграции.
2. Создан API для работы с основными функциями. Код запускается и выполняет требования задания. Допускается и приветствуется любое расширение, дополнение и обоснованное улучшение функционала.
3. Разработан набор тестов для проверки всех эндпойнтов API.
4. Развёртывание: создан файл конфигурации Docker Compose для сборки и запуска проекта в контейнере.

7. Социальная сеть мини-постов

Описание: приложение для обмена короткими записями с возможностью взаимодействия пользователей через лайки и комментарии.

Основные функции:

- Главная лента с записями пользователей.
- Возможность создания, редактирования и удаления собственных записей.
- Система лайков и комментариев.
- Авторизация пользователей.

ИИ-функции:

- Разработайте систему, которая на основе содержания поста будет определять характер содержания (позитивный, негативный, нейтральный), а также возрастные ограничения для каждого поста.

Дополнительные функции:

- Загрузка аватаров пользователей.
- Функция подписки на других пользователей.

Требования к бэкенду:

1. Разработать модель данных для реализации основных функций проекта. Минимальный набор таблиц:
 - пост: идентификатор, заголовок, дата, контент, ссылка на пользователя.

2. Разработать модель данных для работы сервиса регистрации, авторизации и аутентификации пользователей: идентификатор, имя, фамилия, логин, пароль.
3. Разработать API для регистрации, авторизации и аутентификации. Эндпойнты: регистрация, аутентификация, смена пароля, обновление токена.
4. Разработать API для работы основных функций управления постами: создание, удаление, редактирование, чтение (пагинация, фильтры по категориям, дате и пользователю).

Результат реализации бэкенда:

1. Разработана модель данных и настроены миграции.
2. Создан API для работы с основными функциями. Код запускается и выполняет требования задания. Допускается и приветствуется любое расширение, дополнение и обоснованное улучшение функционала.
3. Разработан набор тестов для проверки всех эндпойнтов API.
4. Развёртывание: создан файл конфигурации Docker Compose для сборки и запуска проекта в контейнере.

8. Панель управления задачами команды

Описание: приложение для управления проектами и задачами команды.

Основные функции:

- Доска задач (канбан) с колонками, например «Запланировано», «В работе», «Готово».
- Создание и редактирование задач.
- Возможность добавлять участников к задачам.
- Статистика выполненных задач.

ИИ-функции:

- Создайте систему рекомендаций конкретных задач конкретным пользователям на основе уже решённых этими пользователями задач.

Дополнительные функции:

- Система комментариев в задачах.

- Уведомления о новых задачах и изменениях.

Требования к бэкенду:

1. Разработать модель данных для реализации основных функций проекта. Минимальный набор таблиц:
 - задача: идентификатор, заголовок, дата, описание, тип, ссылка на автора, ссылка на список пользователей, статус.
2. Разработать модель данных для работы сервиса регистрации, авторизации и аутентификации пользователей: идентификатор, имя, фамилия, логин, пароль.
3. Разработать API для регистрации, авторизации и аутентификации. Эндпойнты: регистрация, аутентификация, смена пароля, обновление токена.
4. Разработать API для работы основных функций управления задачами: создание, удаление, редактирование, чтение (пагинация, фильтры по типу, дате, автору, статусу).

Результат реализации бэкенда:

1. Разработана модель данных и настроены миграции.
2. Создан API для работы с основными функциями. Код запускается и выполняет требования задания. Допускается и приветствуется любое расширение, дополнение и обоснованное улучшение функционала.
3. Разработан набор тестов для проверки всех эндпойнтов API.
4. Развёртывание: создан файл конфигурации Docker Compose для сборки и запуска проекта в контейнере.

9. Электронная библиотека

Описание: приложение для поиска, чтения и добавления книг.

Основные функции:

- Поиск книг по названию, автору и категории.
- Страница книги с описанием, отзывами и рейтингом.
- Добавление книг в «Избранное».
- Личный кабинет пользователя для отслеживания прочитанных книг.

ИИ-функции:

- Система рекомендаций книг на основе прочитанной пользователем литературы.

Дополнительные функции:

- Возможность оставлять отзывы и выставять оценки.
- Интеграция с внешними API, например, Google Books API.

Требования к бэкенду:

1. Разработать модель данных для реализации основных функций проекта. Минимальный набор таблиц:
 - автор: идентификатор, ФИО, год рождения, жанры, категории и подобное;
 - категория: идентификатор, наименование категории;
 - книга: идентификатор, название, год издания, ссылка на автора;
 - избранное: ссылка на пользователя, ссылка на книгу, статус прочтения и подобное.
2. Разработать модель данных для работы сервиса регистрации, авторизации и аутентификации пользователей: идентификатор, имя, фамилия, логин, пароль.
3. Разработать API для регистрации, авторизации и аутентификации. Эндпойнты: регистрация, аутентификация, смена пароля, обновление токена.
4. Разработать API, обеспечивающий работу основных функций для работы с книгами: создание, удаление, редактирование, чтение (пагинация, фильтры по категории, дате, автору).
5. Разработать API, обеспечивающий работу основных функций для работы с избранным: добавление в избранное, удаление из избранного, просмотр и подобное.

Результат реализации бэкенда:

1. Разработана модель данных и настроены миграции.
2. Создан API для работы с основными функциями. Код запускается и выполняет требования задания. Допускается и приветствуется любое расширение, дополнение и обоснованное улучшение функционала.
3. Разработан набор тестов для проверки всех эндпойнтов API.
4. Развёртывание: создан файл конфигурации Docker Compose для сборки и запуска проекта в контейнере.

10. Электронный дневник успеваемости

Описание: приложение для автоматизированного расчёта рейтинга успеваемости студентов.

Основные функции:

- Отображение рейтинга успеваемости студентов в рамках группы, курса и отдельных дисциплин.
- Отображение детализированной информации о студенте, группе и курсе.
- Личный кабинет пользователя с доступом к персонализированным данным.

ИИ-функции:

- Создайте систему, которая составляет индивидуальный план занятий для поддержания успеваемости на основе прогресса конкретного студента.

Дополнительные функции:

- Несколько уровней доступа для различных категорий пользователей.
- Поддержка тёмной и светлой тем.

Требования к бэкенду:

1. Разработать модель данных для реализации основных функций проекта. Минимальный набор таблиц:
 - факультет: идентификатор, название;
 - группа: идентификатор, номер, ссылка на факультет, курс;
 - студент: идентификатор, персональные данные, ссылка на таблицу успеваемости;
 - таблица успеваемости: идентификатор, идентификатор студента, дисциплина, текущий бал студента, проходной бал по дисциплине, общее количество занятий, количество оставшихся занятий.
2. Разработать модель данных для работы сервиса регистрации, авторизации и аутентификации пользователей: идентификатор, имя, фамилия, логин, пароль.

3. Разработать API для регистрации, авторизации и аутентификации.
Эндпойнты: регистрация, аутентификация, смена пароля, обновление токена.
4. Разработать API, обеспечивающий работу основных функций для работы с факультетами, группами и студентами: создание, удаление, редактирование, чтение (пагинация, фильтры).
5. Разработать API, обеспечивающий работу основных функций для работы с успеваемостью: выставление оценок, аналитика успеваемости.
6. Разработать эндпойнт для прогноза успеваемости по студенту.
Эндпойнт должен рассчитывать прогноз по дисциплине для конкретного студента и в случае выявления дисциплин с низкими показателями успеваемости, система должна возвращать их список.
Пример: если общее количество занятий по предмету — 10 и проходной балл — 100, то, когда до конца дисциплины осталось 5 занятий, а у студента набрано меньше 50 баллов, система возвращает название дисциплины, как дисциплину с негативным прогнозом.
7. Разработать эндпойнт, позволяющий вывести список студентов с риском неуспеваемости по курсу и факультету.

Результат реализации бэкенда:

1. Разработана модель данных и настроены миграции.
2. Создан API для работы с основными функциями. Код запускается и выполняет требования задания. Допускается и приветствуется любое расширение, дополнение и обоснованное улучшение функционала.
3. Разработан набор тестов для проверки всех эндпойнтов API.
4. Развёртывание: создан файл конфигурации Docker Compose для сборки и запуска проекта в контейнере.